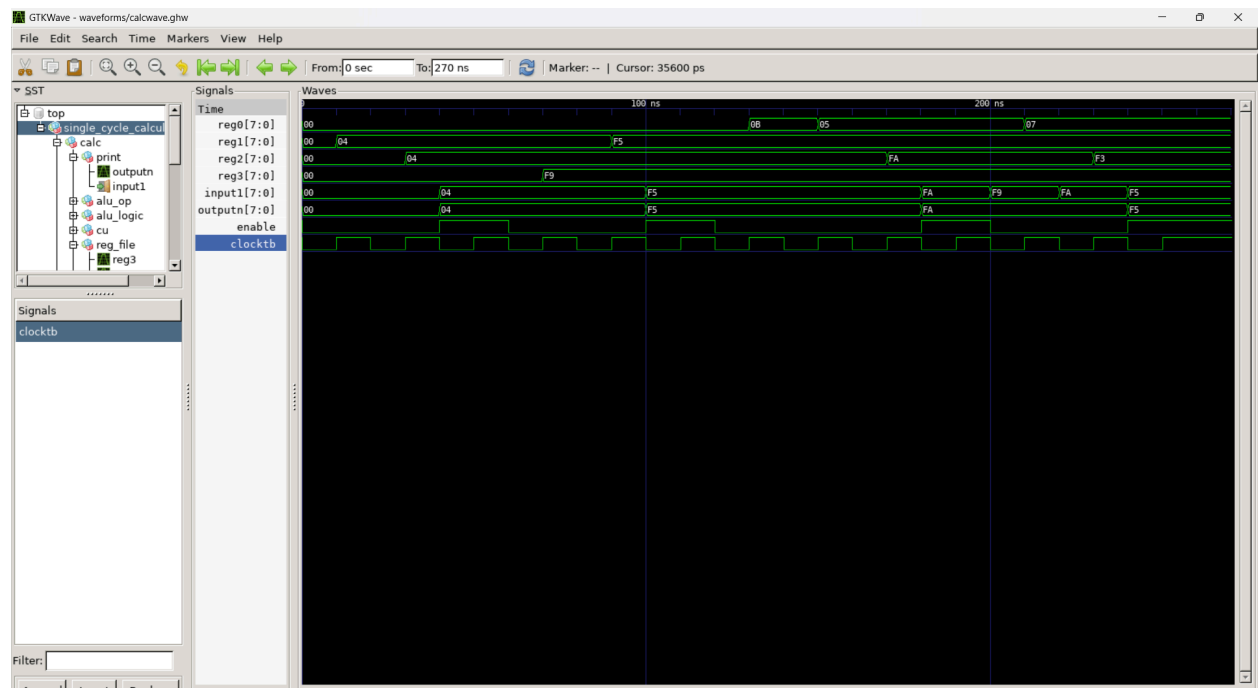


Gerald Stanford

Lab 3 - Lab Report

Test Results

```
components/print_module.vhdl:19:13:@40ns:(report note): 0  
components/print_module.vhdl:19:13:@100ns:(report note): 4  
components/print_module.vhdl:19:13:@180ns:(report note): -11  
components/print_module.vhdl:19:13:@240ns:(report note): -6  
gtkwave waveforms/calcwave.ghw
```



Testing Methodology

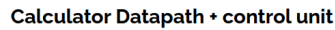
Each component was made as an individual entity and then tested for values I assumed would be used in practice when combined to form the calculator. The assumptions were made based on the truth tables and the general idea of how the component should perform. In vhdl asserts with severity of error were used to get an idea of why a component was performing poorly and severity was set to failure just to ensure the entity was functioning properly.

Test Bench

This is the new test bench used to test the current implementation of the calculator. It is based on the revised ISA submitted after lab 1 but is changed due to how the instructions interact with the ALU unit. The add function became multipurpose and the original test bench didn't account for that.

The test proves that while the calculator works could be improved.

```
1  -- test bench used in the final implementation minus duplication for clock
2
3  load 4 01      11010001  r0=0; r1=4; r2=0; r3=0;
4  add 01 00 10   10010010  r0=0; r1=4; r2=4; r3=0;
5  print 10       00111000  print r2=4
6  load 9 11      11100111  r0=0; r1=4; r2=4; r3=-7; //sign extended
7  add 11 01 01   10110101  r0=0; r1=-11; r2=4; r3=-7; // subtract r3-r1
8  priint 01      00110100  print r1=-11
9  neg 01 00      00100100  r0=11; r1=-11; r2=4; r3=-7;
10 load 5 00      11010100  r0=5; r1=-11; r2=4; r3=-7;
11 add 00 01 10   10000110  r0=5; r1=-11; r2=-6; r3=-7;
12 print 10       00111000  print r2=-6;
13 neg 11 11      00101100  r0=5; r1=-11; r2=-6; r3=7;
14 add 11 10 01   10111010  r0=5; r1=-11; r2=-6; r3=7; // set on equal
15 print 01       00110100  // returns 6 not sure why?
```



The circuit schema was based on the MIPS architecture for the single-cycle cpu. A lot of focus was built around how to properly interact with the alu and implement the arithmetic operations properly. So alot of time was spend figuring out how to perform two's complement and what the alu logic should be doing for a given instruction.